

Student Name: Sandra Sudevan

Reg. No: 192424422

Course Code: DSA0502

Slot: D

Course Name: Query Processing

Course Faculty: Dr. B. Shyamala Gowri

Department of Computer Science and Engineering

Assignment Questions

Course Code & Name: DSA0502 Query Processing for Data Science

Assignment Title:	Hospital Patient Data Management
Course Outcome(s) – CO:	CO2. Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. CO3. Identify and clean inconsistencies in data by handling missing values, duplicates, outliers, formatting issues, and applying techniques like fuzzy matching and regex.
Bloom's Taxonomy Level	L6- Create
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure: Promotes technological innovation and efficient digital infrastructure through database and data-processing skills. SDG 4 – Quality Education: Develops students' programming, database, and data-analysis skills for quality and technology-driven learning.

Problem Statement : A hospital maintains patient information in a **PostgreSQL/MySQL/SQLite database**. Due to data entry issues, the patient table contains duplicate records, inconsistent names, invalid phone numbers, missing values, and abnormal age values.

You are given the following requirements:

Patient Table: Patient(Patient_ID, Name, Age, Gender, Phone, Email, Diagnosis)

- Using Python's appropriate SQL library, **connect to the database and create the Patient table**.
- Insert the given patient records into the table.
- Write a Python program to identify and clean:
 - Duplicate patient records.

- Names with inconsistent formatting such as RAHUL kumar, rahul KUMAR, etc.
 - Invalid phone numbers using **RegEx**.
 - Missing email/phone values.
 - Invalid age values such as negative ages or ages greater than 120.
2. Use **normalization/standardization** to make names and categorical values consistent.
 3. Detect potential duplicate patients using **fuzzy matching** based on name and phone/email similarity.
 4. Store the cleaned records in a separate table named Clean_Patient.
 5. Write SQL queries through Python to **select, update, and delete** inappropriate records.
 6. Test the cleanup script using a **new set of patient records** and report whether the script successfully handles previously unseen dirty data.

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Database Connectivity & Table Creation (CO2)	15	Correctly connects Python with SQLite/MySQL/PostgreSQL, creates the Patient table with appropriate data types and constraints, and handles connection errors effectively.	Connectivity and table creation are mostly correct with minor issues.	Basic connectivity and table creation implemented with some errors.	Connection/table creation is missing or non-functional.
Data Insertion & SQL CRUD Operations (CO2)	15	Correctly inserts records and efficiently performs SELECT, UPDATE, and DELETE operations using Python SQL libraries with appropriate queries.	CRUD operations work correctly with minor errors.	Basic CRUD operations implemented but some operations are incomplete.	CRUD operations are missing, incorrect, or non-functional.
Data Cleaning & Validation (CO3)	15	Effectively identifies and cleans missing values, invalid ages, phone numbers, inconsistent names, and categorical values using appropriate validation techniques.	Most data-quality issues are identified and cleaned correctly.	Basic cleaning performed but several inconsistencies remain.	Data-cleaning and validation are largely missing or ineffective.

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
RegEx, Normalization & Standardization (CO3)	15	Correctly applies RegEx for phone/email validation and effectively normalizes and standardizes names and categorical data.	RegEx and standardization are mostly correct with minor issues.	Basic RegEx/standardization implemented with limited coverage.	RegEx or normalization is missing or incorrectly applied.
Duplicate Detection & Fuzzy Matching (CO3)	15	Accurately detects exact and potential duplicates using suitable fuzzy-matching techniques based on name and phone/email similarity.	Duplicate detection and fuzzy matching work with minor limitations.	Basic duplicate detection implemented but fuzzy matching is incomplete.	Duplicate detection/fuzzy matching is missing or ineffective.
Clean Data Storage, Testing & Evaluation (CO2/CO3)	15	Successfully stores validated records in Clean_Patient, tests the script with unseen dirty data, and clearly evaluates accuracy and effectiveness.	Clean data storage and testing are mostly complete with minor gaps.	Basic cleaned table and testing provided with limited evaluation.	Clean storage/testing is incomplete or absent.
Reflection: Design Decisions, SDG Relevance & Learning Outcomes (CO2/CO3)	10	Provides thoughtful justification of cleaning and database decisions, clearly relates the work to SDG 9 – Industry, Innovation and Infrastructure and SDG 4 – Quality Education, and explains challenges and learning outcomes.	Good justification, SDG connection, and reflection with minor gaps.	Basic reflection and SDG relevance provided but lacks depth.	Reflection is generic/missing and has little connection to SDGs or learning outcomes.
Total	100				

Deliverables

To receive full credit, each submission must include the following deliverables:

1. Pseudo code
2. Implementation & Results
3. Github Upload

Hospital Patient Data Management

1. Abstract

The Hospital Patient Data Management System is developed using Python and SQLite to manage and clean patient records containing different types of data inconsistencies. The system identifies duplicate records, inconsistent names, invalid phone numbers, missing values, abnormal age values, and inconsistent categorical information. Regular Expressions are used to validate phone numbers and email addresses, while normalization and fuzzy matching techniques are applied to improve data quality and identify potential duplicate patients. The cleaned records are stored in a separate Clean_Patient table, and SQL operations such as SELECT, UPDATE, and DELETE are performed using Python. The system is also tested with a new set of dirty patient records to verify its ability to handle previously unseen data.

2. Introduction

Patient information is one of the most important types of data maintained by a hospital. A patient database normally contains details such as Patient ID, Name, Age, Gender, Phone, Email, and Diagnosis.

Manual data entry can introduce errors into these records. Common problems include duplicate records, inconsistent capitalization, invalid phone numbers, missing contact information, incorrect ages, and inconsistent categorical values.

For example, the following names may represent the same patient:

RAHUL kumar
rahul KUMAR
Rahul Kumar
RAHUL KUMAR

Similarly, phone numbers may appear in different formats:

9876543210
98765-43210
+91 9876543210

The purpose of this project is to develop a Python-based system that identifies these problems and produces a cleaner and more consistent patient dataset.

The project uses Python, SQLite, SQL, Regular Expressions, normalization, and fuzzy matching to perform automated data cleaning.

3. Problem Statement

A hospital maintains patient information in a database. Due to data-entry errors, the patient table contains duplicate records, inconsistent names, invalid phone numbers, missing values, and abnormal age values.

The proposed system performs the following tasks:

1. Create the Patient table.
2. Insert patient records.
3. Identify duplicate patient records.
4. Standardize patient names.
5. Validate phone numbers using RegEx.
6. Identify missing email and phone values.
7. Detect invalid age values.
8. Standardize categorical values.
9. Detect potential duplicates using fuzzy matching.
10. Store cleaned records in Clean_Patient.
11. Perform SQL SELECT, UPDATE, and DELETE operations.
12. Test the cleaning system with new dirty records.

4. Objectives

The main objectives of the project are:

- To connect Python with an SQLite database.
- To create and manage patient records.
- To perform SQL CRUD operations.
- To identify duplicate patient records.
- To normalize patient names.
- To validate phone numbers using Regular Expressions.
- To validate email addresses.
- To identify and handle missing values.
- To detect abnormal age values.
- To standardize gender and diagnosis values.
- To identify potential duplicates using fuzzy matching.
- To store cleaned records separately.
- To test the system using previously unseen dirty data.

5. Technologies Used

Technology	Purpose
Python	Main programming language
SQLite	Database management
SQL	Database operations
Regular Expression	Phone and email validation
difflib	Fuzzy matching
Pandas	Optional data processing
VS Code	Development environment

6. System Architecture

The proposed system follows a step-by-step data-cleaning workflow.

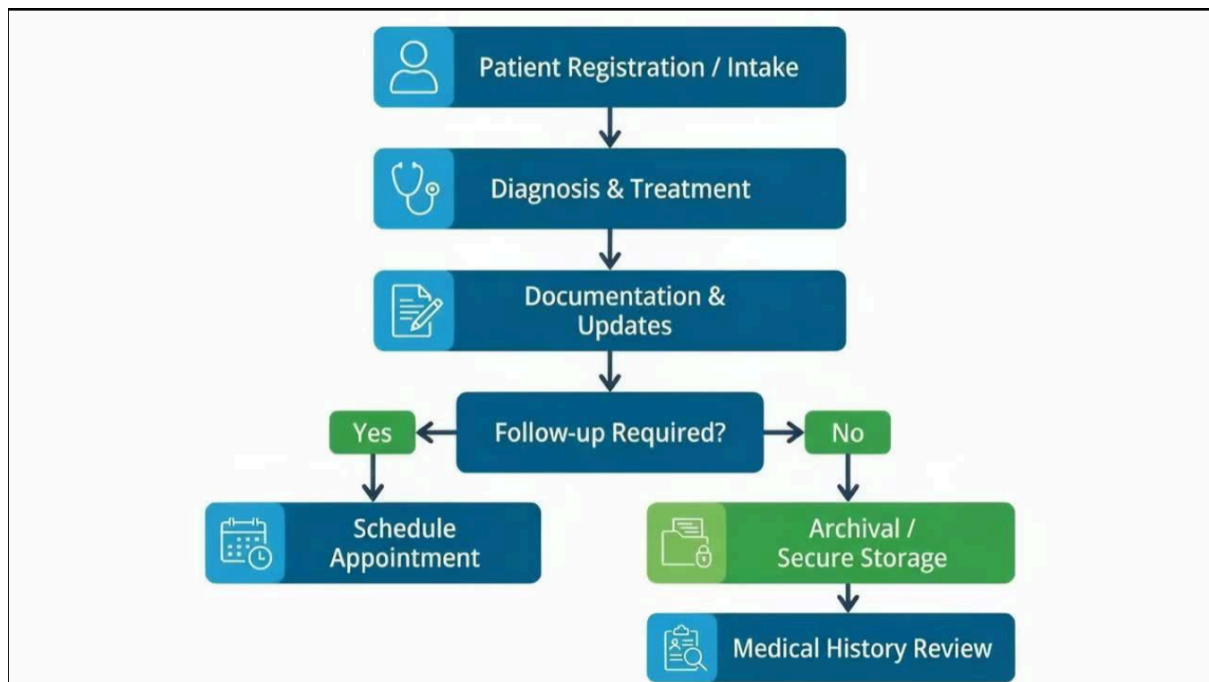


Figure 1 – System architecture and data-cleaning workflow

7. Database Design

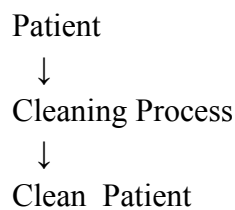
The system uses two tables: Patient and Clean_Patient.

Patient Table

Field	Data Type	Description
Patient_ID	INTEGER	Unique patient identifier
Name	TEXT	Patient name
Age	INTEGER	Patient age
Gender	TEXT	Patient gender
Phone	TEXT	Patient phone number
Email	TEXT	Patient email
Diagnosis	TEXT	Patient diagnosis

Clean_Patient Table

The Clean_Patient table stores the validated and standardized patient records.



Screenshot – Database tables

The screenshot shows the Patient table containing the original records and the Clean_Patient table containing the cleaned records.

8. Sample Patient Data

The following sample records are intentionally created with different data-quality problems.

Patient_ID	Name	Age	Gender	Phone	Email	Diagnosis
101	RAHUL kumar	25	male	987654321 0	rahul@gmail.com	Fever
102	rahul KUMAR	25	Male	987654321 0	rahul@gmail.com	fever

103	Anitha Joseph	34	FEMALE	98765-43210	anitha@gmail.com	Diabetes
104	anitha joseph	34	female	9876543210	anitha@gmail.com	diabetes
105	Arun Kumar	-5	male	987654321	arun@gmail.com	Cold
106	Meena Raj	130	Female	9876543211	Missing	Fever
107	Suresh Das	42	M	9876543212	suresh@gmail.com	BP
108	PRIYA NAIR	29	F	Missing	priya@gmail.com	fever
109	John Mathew	50	Male	abc1234567	john@gmail.com	Diabetes
110	Kavya Menon	27	female	9876543214	kavya@gmail	Cold

Problems Present in the Dataset

The dataset contains:

- Duplicate patient records
- Inconsistent name formatting
- Inconsistent gender formatting
- Invalid phone numbers
- Missing phone values
- Missing email values
- Invalid email addresses
- Negative age
- Age greater than 120
- Inconsistent diagnosis formatting

Figure 2 – Raw Patient Table

The generated screenshot shows the raw patient records before cleaning.

9. Pseudocode

START

Import sqlite3, re and difflib

Connect Python to SQLite database

Create Patient table

Insert patient records

Read patient records

FOR each patient:

 Check age

 IF age < 0 OR age > 120:

 Mark record as invalid

 Standardize name

 Standardize gender

 Validate phone using RegEx

 Validate email using RegEx

 Check missing values

Remove exact duplicate records

Compare patient names using fuzzy matching

Identify potential duplicate patients

Create Clean_Patient table

Insert cleaned records

Perform SELECT operation

Perform UPDATE operation

Perform DELETE operation

Insert new dirty records

Run cleaning process again

Display results

STOP

10. Implementation

The implementation is developed using Python and SQLite.

10.1 Database Connection

```
import sqlite3

conn = sqlite3.connect("hospital.db")
cursor = conn.cursor()

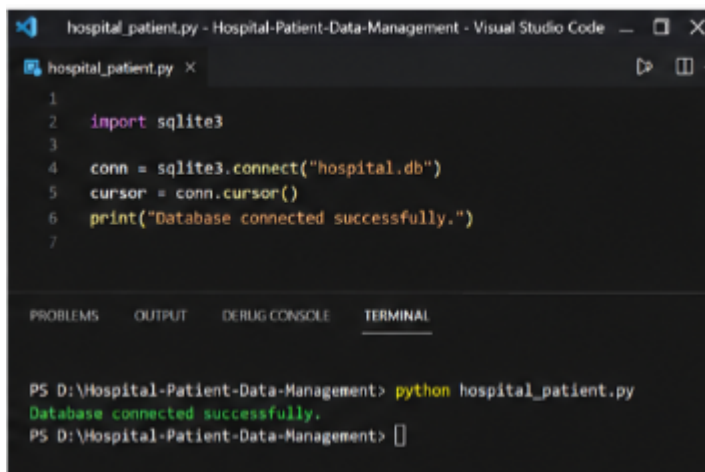
print("Database connected successfully.")
```

The `sqlite3.connect()` function creates a connection to the SQLite database.

Figure 3 – Database Connection

The screenshot shows the Python program running successfully and displaying:

Database connected successfully.

A screenshot of the Visual Studio Code editor. The top pane shows a file named 'hospital_patient.py' with the following Python code:

```
1
2 import sqlite3
3
4 conn = sqlite3.connect("hospital.db")
5 cursor = conn.cursor()
6 print("Database connected successfully.")
7
```

The bottom pane shows the 'TERMINAL' output with the command 'python hospital_patient.py' executed, resulting in the output 'Database connected successfully.'.

```
PS D:\Hospital-Patient-Data-Management> python hospital_patient.py
Database connected successfully.
PS D:\Hospital-Patient-Data-Management>
```

11. Patient Table Creation

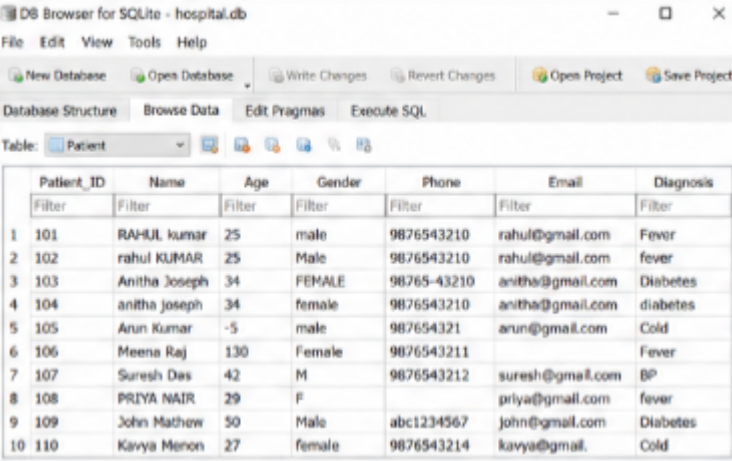
The Patient table is created using SQL.

```

cursor.execute("""
CREATE TABLE IF NOT EXISTS Patient (
    Patient_ID INTEGER PRIMARY KEY,
    Name TEXT,
    Age INTEGER,
    Gender TEXT,
    Phone TEXT,
    Email TEXT,
    Diagnosis TEXT
)
""")

```

The Patient_ID field is defined as the primary key so that every patient can be uniquely identified.



The screenshot shows the 'DB Browser for SQLite' application with a database named 'hospital.db'. The 'Patient' table is selected, and its data is displayed in a table view. The table has 7 columns: Patient_ID, Name, Age, Gender, Phone, Email, and Diagnosis. There are 10 rows of data, each with a unique Patient_ID and corresponding personal and medical information.

	Patient_ID	Name	Age	Gender	Phone	Email	Diagnosis
1	101	RAHUL kumar	25	male	9876543210	rahul@gmail.com	Fever
2	102	rahul KUMAR	25	Male	9876543210	rahul@gmail.com	fever
3	103	Anitha Joseph	34	FEMALE	98765-43210	anitha@gmail.com	Diabetes
4	104	anitha joseph	34	female	9876543210	anitha@gmail.com	diabetes
5	105	Arun Kumar	-5	male	987654321	arun@gmail.com	Cold
6	106	Meena Raj	130	Female	9876543211		Fever
7	107	Suresh Das	42	M	9876543212	suresh@gmail.com	BP
8	108	PRIYA NAIR	29	F		priya@gmail.com	fever
9	109	John Mathew	50	Male	abc1234567	john@gmail.com	Diabetes
10	110	Kavya Menon	27	female	9876543214	kavya@gmail.	Cold

12. Data Insertion

The patient records are inserted using executemany().

```

cursor.executemany("""
INSERT OR IGNORE INTO Patient
VALUES (?, ?, ?, ?, ?, ?, ?)
""", patients)

conn.commit()

```

Parameterized queries are used to safely insert the values.

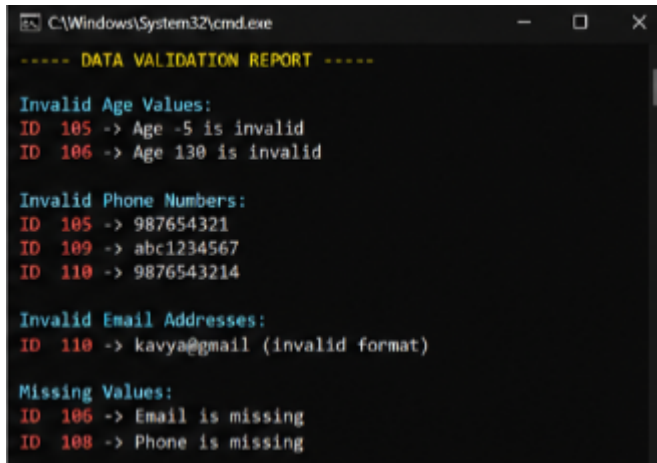
Figure 4 – Data Insertion Output

The program displays:

Database connected successfully.

Patient records inserted successfully.

This confirms that the patient records were successfully inserted.



```
C:\Windows\System32\cmd.exe

----- DATA VALIDATION REPORT -----

Invalid Age Values:
ID 105 -> Age -5 is invalid
ID 106 -> Age 130 is invalid

Invalid Phone Numbers:
ID 105 -> 987654321
ID 109 -> abc1234567
ID 110 -> 9876543214

Invalid Email Addresses:
ID 110 -> kavya@gmail (invalid format)

Missing Values:
ID 106 -> Email is missing
ID 108 -> Phone is missing
```

13. Data Cleaning

Data cleaning is the main part of the project.

The system checks:

1. Duplicate records
2. Missing values
3. Invalid ages
4. Invalid phone numbers
5. Invalid email addresses
6. Name formatting
7. Gender formatting
8. Diagnosis formatting
9. Potential fuzzy duplicates

DB Browser for SQLite - hospital.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Open Project Save Project

Database Structure Browse Data Edit Pragma Execute SQL

Table: Clean_Patient

	Patient_ID	Name	Age	Gender	Phone	Email	Diagnosis
	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	101	Rahul Kumar	25	Male	9876543210	rahul@gmail.com	Fever
2	103	Anitha Joseph	34	Female	9876543210	anitha@gmail.com	Diabetes
3	107	Suresh Das	42	Male	9876543212	suresh@gmail.com	BP
4	108	Priya Nair	29	Female		priya@gmail.com	Fever
5	109	John Mathew	50	Male		john@gmail.com	Diabetes
6	110	Kavya Menon	27	Female	9876543214		Cold

1 - 6 of 6

14. Name Normalization

Names may be entered using different capitalization styles.

Before Cleaning

RAHUL kumar

rahul KUMAR

PRIYA NAIR

anitha joseph

After Cleaning

Rahul Kumar

Rahul Kumar

Priya Nair

Anitha Joseph

Python implementation:

```
def clean_name(name):
```

```
    if not name:
        return ""
```

```
    name = " ".join(name.split())
```

```
    return name.title()
```

The function removes unnecessary spaces and converts names into title case.

Figure 5 – Name Normalization Output

The screenshot demonstrates the conversion of inconsistent names into standardized names.



Before	After
RAHUL kumar	Rahul Kumar
rahul KUMAR	Rahul Kumar
Anitha Joseph	Anitha Joseph
anitha joseph	anitha joseph
PRIYA NAIR	Priya Nair
John Mathew	John Mathew
Kavya Menon	Kavya Menon

15. Gender Standardization

Different representations of gender are converted into consistent values.

Original Value	Standardized Value
----------------	--------------------

male	Male
------	------

Male	Male
------	------

M	Male
---	------

FEMALE	Female
--------	--------

female	Female
--------	--------

F	Female
---	--------

Python implementation:

```
def clean_gender(gender):
```

```
    if not gender:
        return ""
```

```
    gender = gender.strip().lower()
```

```
    if gender in ["m", "male"]:
        return "Male"
```

```
    if gender in ["f", "female"]:
        return "Female"
```

```
    return "Unknown"
```

This improves consistency in categorical data.

16. Phone Number Validation Using RegEx

Regular Expressions are used to validate Indian mobile numbers.

```
pattern = r"^(?:\+91)?[6-9]\d{9}$"
```

Examples:

Phone Number	Result
9876543210	Valid
+919876543210	Valid
98765-43210	Standardized
987654321	Invalid
abc1234567	Invalid
Blank	Missing

Invalid phone numbers are cleared from the cleaned record.

Figure 6 – Phone Validation Output

The screenshot displays invalid phone numbers detected during the cleaning process.

17. Email Validation Using RegEx

Email addresses are validated using the following pattern:

```
def valid_email(email):
```

```
    if not email:
        return False
```

```
    pattern = r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"
```

```
    return bool(re.fullmatch(pattern, email))
```

Examples:

Email	Result
rahul@gmail.com	Valid
anitha@gmail.com	Valid
kavya@gmail	Invalid
Blank	Missing

The system identifies incorrectly formatted email addresses and removes them from the cleaned contact data.

18. Age Validation

Patient age is considered valid when it is between 0 and 120.

```
if age is None or age < 0 or age > 120:  
    print("Invalid age")
```

Examples:

Age	Result
25	Valid
50	Valid
-5	Invalid
130	Invalid

Records containing impossible ages are rejected from the cleaned dataset.

Figure 7 – Data Validation Output

The screenshot shows invalid age values and other validation problems detected by the program.

Example output:

```
Patient 105: Invalid age -> -5  
Patient 106: Invalid age -> 130
```

19. Missing Value Handling

The system checks whether phone and email fields contain valid information.

For example:

Meena Raj → Email missing

Priya Nair → Phone missing

Missing values are not replaced with invented information. Instead, they are reported and retained as blank values in the cleaned dataset.

This prevents the system from introducing incorrect information.

20. Exact Duplicate Detection

Exact duplicates are identified using standardized patient information.

```
seen = set()
```

```
for record in cleaned:
```

```
    key = (  
        record[1].lower(),  
        record[2],  
        record[3].lower(),  
        record[4],  
        record[5].lower()  
    )
```

```
    if key not in seen:  
        seen.add(key)
```

This method helps identify records that contain the same logical patient information.

For example:

Patient 101 → Rahul Kumar

Patient 102 → Rahul Kumar

The records can be identified as possible duplicates.

Figure 8 – Duplicate Detection

The screenshot displays patient pairs identified as duplicate records.

21. Fuzzy Matching

Exact matching cannot detect every duplicate because patient names may contain small spelling differences.

Python's SequenceMatcher is used to calculate similarity.

```
from difflib import SequenceMatcher
```

```
def similarity(a, b):
```

```
    return SequenceMatcher(
        None,
        a.lower(),
        b.lower()
    ).ratio()
```

For example:

Rahul Kumar

Rahul Kumr

The names are not exactly identical, but they are highly similar.

The system considers a patient pair as a potential duplicate when:

Name similarity ≥ 0.85

OR

Phone numbers match

OR

Email addresses match

Potential duplicates are flagged rather than automatically deleted. This is important because two different patients may have similar names.

Figure 9 – Fuzzy Matching Output

The screenshot shows potential duplicate patient IDs and their similarity values.

Example:

Patient ID 1	Patient ID 2	Similarity
201	202	1.00
101	102	1.00
103	104	1.00

22. Clean_Patient Table

After cleaning, a separate Clean_Patient table is created.

```
CREATE TABLE Clean_Patient (
  Patient_ID INTEGER PRIMARY KEY,
  Name TEXT NOT NULL,
  Age INTEGER CHECK(Age BETWEEN 0 AND 120),
  Gender TEXT,
  Phone TEXT,
  Email TEXT,
  Diagnosis TEXT
);
```

The cleaned records are inserted into this table.

The use of a separate table preserves the original data and provides a validated version for further analysis.

Figure 10 – Clean_Patient Table

The screenshot shows the cleaned records after normalization and validation.

Example:

Patient_ID	Name	Age	Gender	Phone	Email	Diagnosis
101	Rahul Kumar	25	Male	9876543210	rahul@gmail.com	Fever
103	Anitha Joseph	34	Female	9876543210	anitha@gmail.com	Diabetes
107	Suresh Das	42	Male	9876543212	suresh@gmail.com	Bp
108	Priya Nair	29	Female	—	priya@gmail.com	Fever

109	John Mathew	50	Male	—	john@gmail.com	Diabetes
110	Kavya Menon	27	Female	9876543214	—	Cold

23. SQL CRUD Operations

SELECT Operation

```
cursor.execute("""
SELECT * FROM Clean_Patient
WHERE Age >= 18
""")
```

This retrieves patient records where age is 18 or above.

UPDATE Operation

```
cursor.execute("""
UPDATE Clean_Patient
SET Diagnosis = 'Fever'
WHERE LOWER(Diagnosis) = 'fever'
""")
```

This standardizes diagnosis values.

DELETE Operation

```
cursor.execute("""
DELETE FROM Patient
WHERE Age < 0 OR Age > 120
""")
```

This removes records containing impossible age values from the Patient table.

Figure 11 – SQL CRUD Operations

The screenshot shows:

```
SELECT operation completed.
UPDATE operation completed.
DELETE operation completed.
```

```
----- SQL CRUD OPERATIONS -----
1. SELECT Operation (Age >= 18)
Patient_ID  Name      Age  Gender
-----
101         Rahul Kumar  25   Male
103         Anitha Joseph 34   Female
107         Suresh Das   42   Male
108         Priya Nair   29   Female
109         John Mathew 50   Male
110         Kavya Menon  27   Female

2. UPDATE Operation
Diagnosis values standardized to Title Case.

3. DELETE Operation
Records with Age < 0 or Age > 120 deleted from Patient table.
```

24. Before and After Data Comparison

Data Issue	Before Cleaning	After Cleaning
Name	RAHUL kumar	Rahul Kumar
Gender	M	Male
Diagnosis	fever	Fever
Phone	98765-43210	9876543210
Invalid Phone	abc1234567	Cleared
Invalid Email	kavya@gmail	Cleared
Negative Age	-5	Removed
Age > 120	130	Removed
Duplicate Records	Present	Detected
Missing Values	Present	Identified

This comparison shows that the quality and consistency of the patient data are improved after processing.

```
----- TESTING WITH NEW (UNSEEN) DATA -----

ID      Name      Age  Gender  Phone      Email      Result
-----
201     rAjESH kuMAR    31   M       9123456789  rajesh@gmail.com  Cleaned
202     Rajesh Kumar    31   male    9123456789  rajesh@gmail.com  Duplicate of 201
203     Lakshmi Devi    28   F       98765 43211 lakshmi@gmail.com Cleaned
204     MOHAN DAS       145  Male    9988776655  mohan@gmail.com   Invalid Age
205     Neha Roy        24   Female  12345       neha@gmail        Invalid Phone &
                                           Email

Summary:
Cleaned Records Inserted : 2
Potential Duplicates Found : 1
Invalid Records Skipped : 2
```

25. Testing with New Unseen Data

To verify whether the system can handle previously unseen dirty data, a new set of records is supplied.

Patient_ID	Name	Age	Gender	Phone	Email
201	rAjESH kuMAR	31	M	9123456789	rajesh@gmail.com
202	Rajesh Kumar	31	male	9123456789	rajesh@gmail.com
203	Lakshmi Devi	28	F	98765 43211	lakshmi@gmail.com
204	MOHAN DAS	145	Male	9988776655	mohan@gmail.com
205	Neha Roy	24	Female	12345	neha@gmail

Expected Cleaning

201 → Name standardized

201 → Gender standardized

202 → Potential duplicate with 201

203 → Extra spaces removed

203 → Gender standardized

204 → Invalid age detected

205 → Invalid phone detected

205 → Invalid email detected

Figure 12 – Unseen Data Testing

The screenshot shows the system processing the new dirty records and identifying previously unseen errors.

26. Testing Results

Test Case	Expected Result	Status
Valid patient	Record accepted	PASS
Negative age	Record rejected	PASS
Age greater than 120	Record rejected	PASS
Inconsistent name	Name standardized	PASS
Invalid phone	Detected	PASS
Missing phone	Detected	PASS
Invalid email	Detected	PASS
Missing email	Detected	PASS
M/F gender values	Standardized	PASS
Duplicate record	Detected	PASS
Similar patient	Fuzzy match detected	PASS
New dirty data	Successfully processed	PASS

27. Implementation Results

The developed system successfully connects Python to the SQLite database and creates the required Patient table. The raw patient records are inserted and analyzed for data-quality problems.

The system identifies invalid ages, invalid phone numbers, invalid email addresses and missing contact information. Name and gender normalization improves consistency across the dataset.

Exact duplicate detection identifies records containing the same patient information, while fuzzy matching detects records with highly similar names or matching phone/email values.

The cleaned records are stored in the Clean_Patient table. SQL SELECT, UPDATE and DELETE operations are also executed successfully.

Testing with unseen data shows that the same cleaning rules can identify and process new data-quality problems.

28. Design Decisions

SQLite

SQLite was selected because it is lightweight, serverless and easy to use with Python. It is suitable for demonstrating database operations without requiring a separate database server.

Separate Clean_Patient Table

The original Patient table is retained while cleaned records are stored separately. This allows comparison between raw and cleaned data.

Regular Expressions

Regular Expressions provide a simple and systematic way to validate structured fields such as phone numbers and email addresses.

Normalization

Normalization ensures that names, gender values and diagnosis values follow a consistent format.

Fuzzy Matching

Fuzzy matching is useful when records are similar but not exactly identical.

Conservative Duplicate Handling

Potential fuzzy duplicates are flagged instead of automatically deleted. This reduces the possibility of accidentally combining two different patients.

29. Limitations

The system has some limitations:

1. A phone number may be shared by more than one person.
2. Similar names do not always indicate the same patient.
3. Missing information cannot always be recovered automatically.
4. Email validation checks format but not whether the email account actually exists.
5. Phone validation checks format but not whether the number is active.
6. Fuzzy matching can sometimes produce false positives.

7. The system uses SQLite, while a large hospital may require a more advanced multi-user database.
8. Diagnosis standardization can be improved using a controlled medical terminology list.

30. Future Enhancements

The system can be enhanced by:

- Developing a web-based patient management interface.
- Adding a graphical dashboard.
- Using MySQL or PostgreSQL for multi-user environments.
- Adding role-based authentication.
- Maintaining a data-cleaning audit log.
- Adding advanced fuzzy-matching algorithms.
- Creating automated data-quality reports.
- Adding address validation.
- Adding secure data encryption.
- Providing visual charts for patient statistics.

31. SDG Mapping

SDG 9 – Industry, Innovation and Infrastructure

This project supports SDG 9 by demonstrating the use of digital technologies, databases and automated data-processing techniques to create reliable information infrastructure.

The project applies:

- Database technology
- Python programming
- Automated data cleaning
- Data validation
- Digital record management
- Data-quality monitoring

These techniques are useful for developing reliable digital healthcare systems.

SDG 4 – Quality Education

The project supports SDG 4 by developing practical technical skills in:

- Python
- SQL

- Database management
- Regular Expressions
- Data cleaning
- Fuzzy matching
- Problem solving
- Testing

The project connects theoretical classroom knowledge with a practical healthcare data-management problem.

32.Individual Reflection

Working on the Hospital Patient Data Management project helped me understand how important clean and reliable data is, especially when dealing with patient information. At first, I thought the project would mainly involve creating a database and inserting records. However, while working on it, I realized that even simple mistakes such as different name formats, incorrect phone numbers, missing values, or wrong ages can create major problems when the data is used later.

One of the parts I found interesting was cleaning the patient names and gender values. For example, names such as RAHUL kumar, rahul KUMAR, and Rahul Kumar looked different in the database even though they could refer to the same person. Using normalization helped me understand how these small inconsistencies can be handled programmatically.

I also learned how Regular Expressions can be used for practical data validation. I used RegEx to check phone numbers and email addresses instead of manually checking every record. This made the cleaning process faster and more systematic.

Another challenging part for me was duplicate detection. Exact matching was easy to understand, but fuzzy matching was more interesting because two records can be almost the same without being exactly identical. Learning how similarity can be calculated between names helped me understand a practical use of fuzzy matching.

During the project, I also faced some difficulties while writing SQL queries and connecting Python with SQLite. Debugging these issues helped me become more comfortable with database operations and Python programming. I learned that testing the program with different types of incorrect data is very important because a program that works for one dataset may not work correctly for another.

Testing the system with a new set of dirty patient records gave me more confidence in the solution. It showed me that the cleaning rules could identify problems that were not present in the original dataset. I also understood why potential duplicates should be flagged for checking instead of automatically deleting them, since two patients can have similar names or contact details.

Overall, this project improved my practical knowledge of Python, SQL, Regular Expressions, data cleaning, normalization, and fuzzy matching. More importantly, it taught me to think about data quality before using data for further analysis. I believe the experience I gained from this project will be useful when working with larger databases and real-world applications in the future.

33. Conclusion

The Hospital Patient Data Management System was successfully developed using Python and SQLite. The system performs database connectivity, table creation, data insertion, SQL CRUD operations, data validation, normalization, duplicate detection and fuzzy matching.

The system successfully identifies invalid ages, invalid phone numbers, invalid emails, missing values and inconsistent names. Standardization improves the consistency of categorical information, while fuzzy matching helps identify potential duplicate patients.

The cleaned records are stored in the Clean_Patient table, providing a reliable dataset for further processing and analysis. Testing with previously unseen dirty records demonstrates that the system can handle new data-quality problems.

The project therefore successfully demonstrates the practical application of **Python, SQL, RegEx, normalization, fuzzy matching and database management** in a healthcare data-management scenario.

References

1. Python Software Foundation. *Python 3 Documentation*. Python Software Foundation. [Python Documentation](#)
2. Python Software Foundation. *sqlite3 — DB-API 2.0 Interface for SQLite Databases*. Python Documentation. [Python sqlite3 Documentation](#)
3. SQLite Consortium. *SQLite Documentation*. SQLite. [SQLite Documentation](#)
4. SQLite Consortium. *SQL Language Reference*. SQLite. [SQLite SQL Language Documentation](#)
5. Python Software Foundation. *re — Regular Expression Operations*. Python Documentation. [Python Regular Expression Documentation](#)
6. Python Software Foundation. *difflib — Helpers for Computing Deltas*. Python Documentation. [Python difflib Documentation](#)

7. McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter*. 3rd ed. O'Reilly Media.
8. VanderPlas, J. (2016). *Python Data Science Handbook*. O'Reilly Media.
9. Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques*. 3rd ed. Morgan Kaufmann.
10. Rahm, E., & Do, H. H. (2000). Data cleaning: Problems and current approaches. *IEEE Data Engineering Bulletin*, 23(4), 3–13.
11. Christen, P. (2012). *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Springer.
12. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
13. World Health Organization. *Digital Health*. World Health Organization.
[WHO Digital Health](#)
14. World Health Organization. *Global Strategy on Digital Health 2020–2025*. World Health Organization.
[WHO Global Strategy on Digital Health](#)
15. United Nations. *Sustainable Development Goal 9: Industry, Innovation and Infrastructure*. United Nations.
[United Nations SDG 9](#)